

PRACTICAL CRYPTOGRAPHY

crypt@b-it 2026
Summer school on Cryptography
Sofía Celi



FrodoFIR

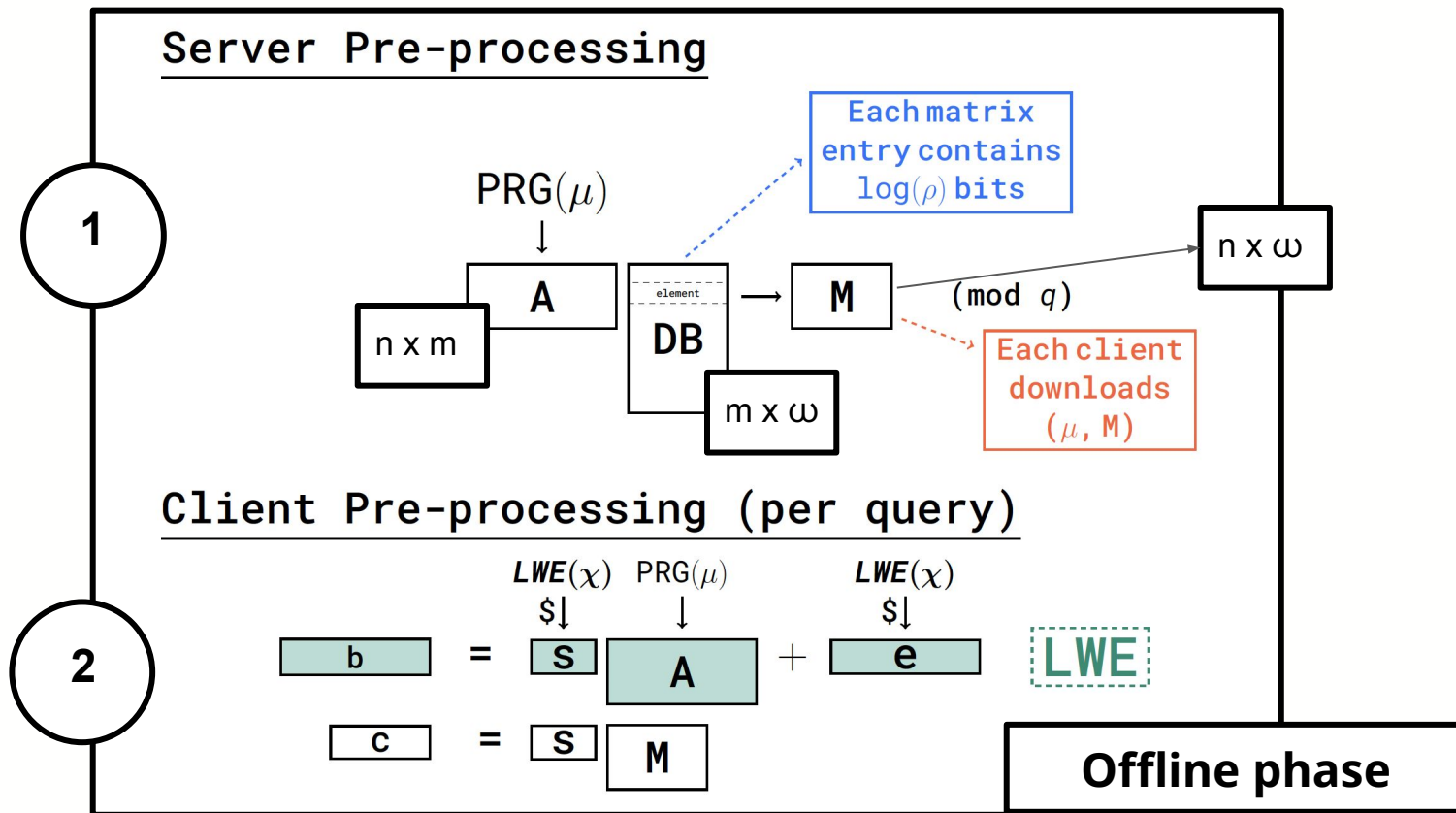
LECTURE 2: PRIVATE INFORMATION RETRIEVAL:

ARE WE CLOSE TO MAKE IT PRACTICAL?

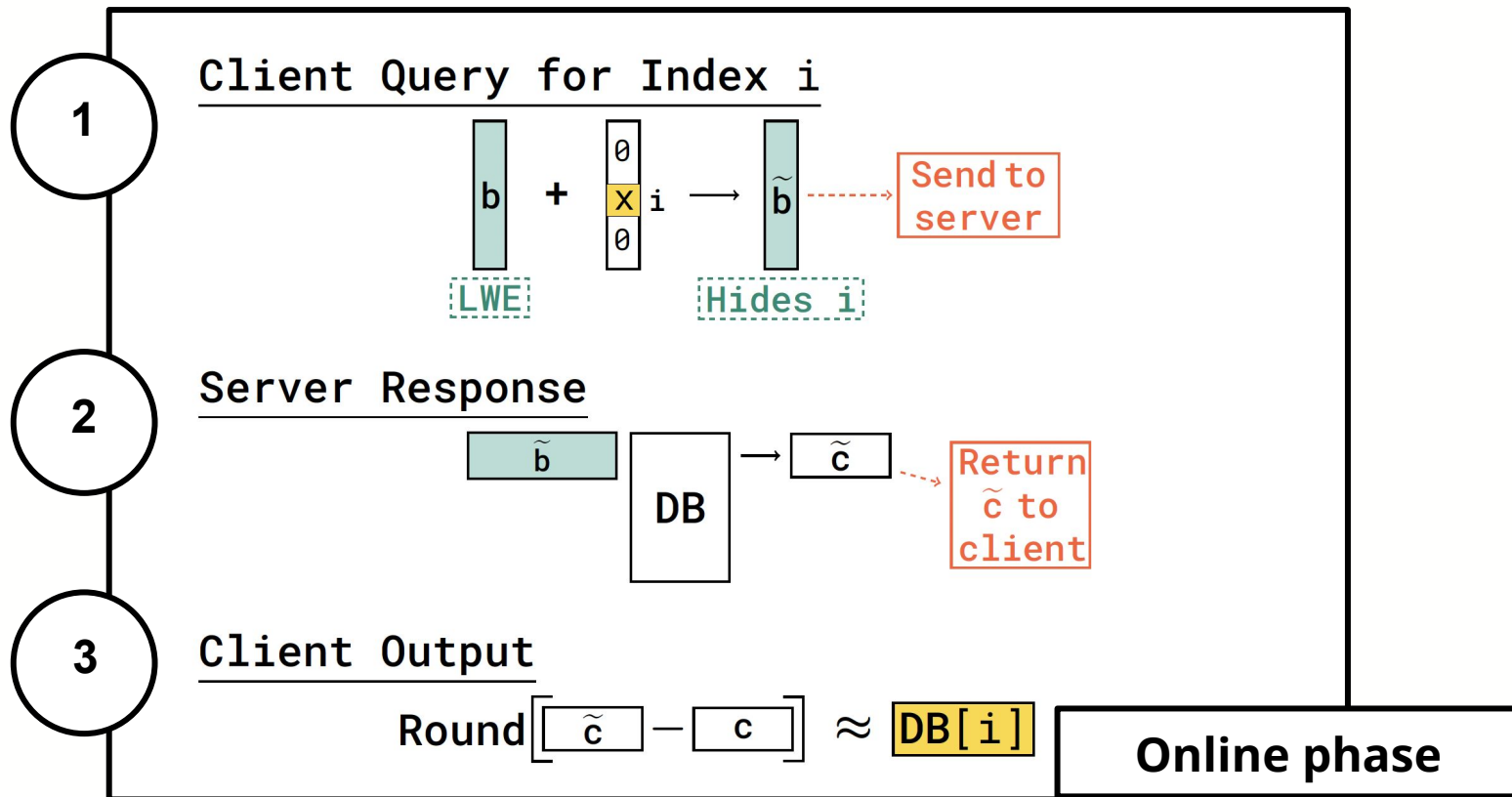


FrodoFIR

FrodoPIR (offline)



FrodoPIR (online)



But, is this enough?

- Databases are not structured in this simple way
 - They are indexed by keywords
 - They are structured as JSON, Graphs, Excel spreadsheets
- The queries we are interested in are not simple:
 - Complex queries with AND/OR statements
 - Combination of database systems
 - Approximate nearest neighbor (ANN) elements
- Databases are constantly updated
- Is the security we assume enough?
 - What about malicious security?
 - What about private databases?

**Not all systems are
created equally**

But, is this enough?

- Databases are not structured in this simple way
 - They are indexed by keywords
 - They are structured as JSON, Graphs, Excel spreadsheets
- The queries we are interested in are not simple
 - Complex queries with AND/OR statements
 - Combination of database systems
 - Approximate nearest neighbor (ANN) elements
- Databases are constantly updated
- Is the security we assume enough?
 - What about malicious security?
 - What about private databases?

How can we handle this?

CHALAMET-PIR

(one solution)

<https://eprint.iacr.org/2024/092>

Keyword-based PIR

- So far the client knows the index i of the record it wants
- Real applications query by *key*: a URL, a domain, a password hash, a phone number
- **Keyword PIR**: the DB is a key-value map, and the client holds a key, not an index
- Naive approaches are costly:
 - Hash the key to an index: collisions
 - Cuckoo hashing: several candidate locations, so several PIR queries
 - PIR over a search tree (Chor–Gilboa–Naor '97): $\log m$ rounds
- Goal: one query, one round, server work unchanged

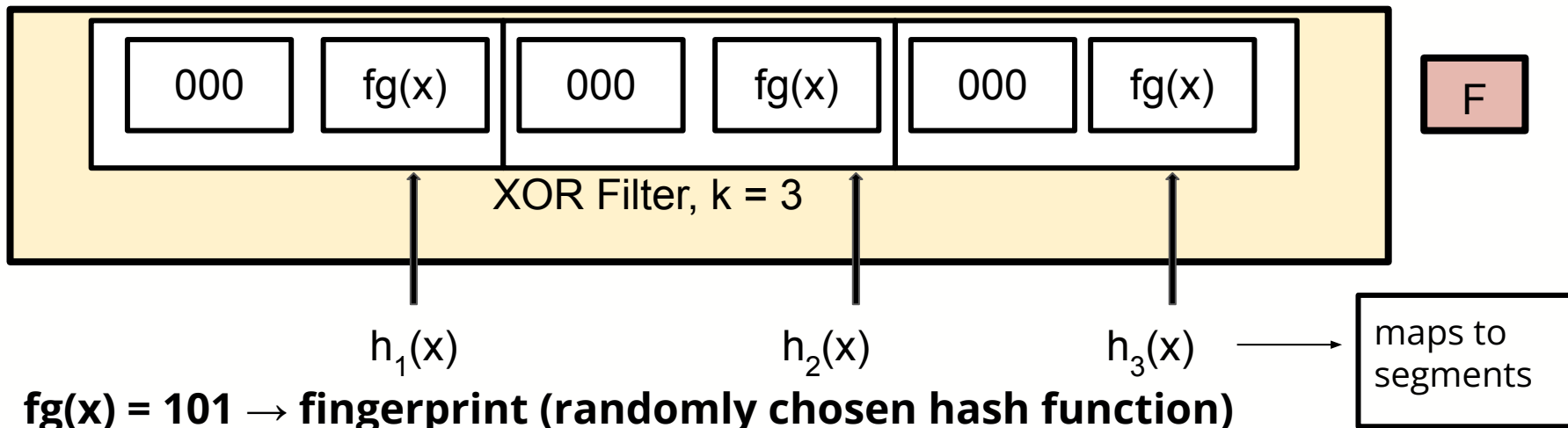
Enter filters...

- A filter is a *compact probabilistic data structure* for approximate set membership
 - No false negatives but false positives occur with probability about 2^{-k} for k -bit fingerprints
 - Bloom (1970) → cuckoo filters → XOR filters (Graf-Lemire '20) → binary fuse filters (Graf-Lemire '22)
- XOR and binary fuse filters store an array F of about $1.08\text{--}1.13 \times m$ entries and use a few public hash functions h_1, h_2, h_3 (or four)
 - F is built so that $F[h_1(x)] \oplus F[h_2(x)] \oplus F[h_3(x)] = \text{fingerprint}(x)$ for every x in the set
 - A lookup reads 3-4 fixed positions, combines them, and compares
- Key-value variant: store the value in place of the fingerprint
 - $F[h_1(\text{key})] + F[h_2(\text{key})] + F[h_3(\text{key})] = \text{value}$
 - The lookup is a linear function of a few positions of F

Binary-Fuse Filters (BFF) [ML22]

- Like a XOR filter [ML20]:
 - Items are known in advance, it is immutable
 - False negatives are not allowed, but false positives are

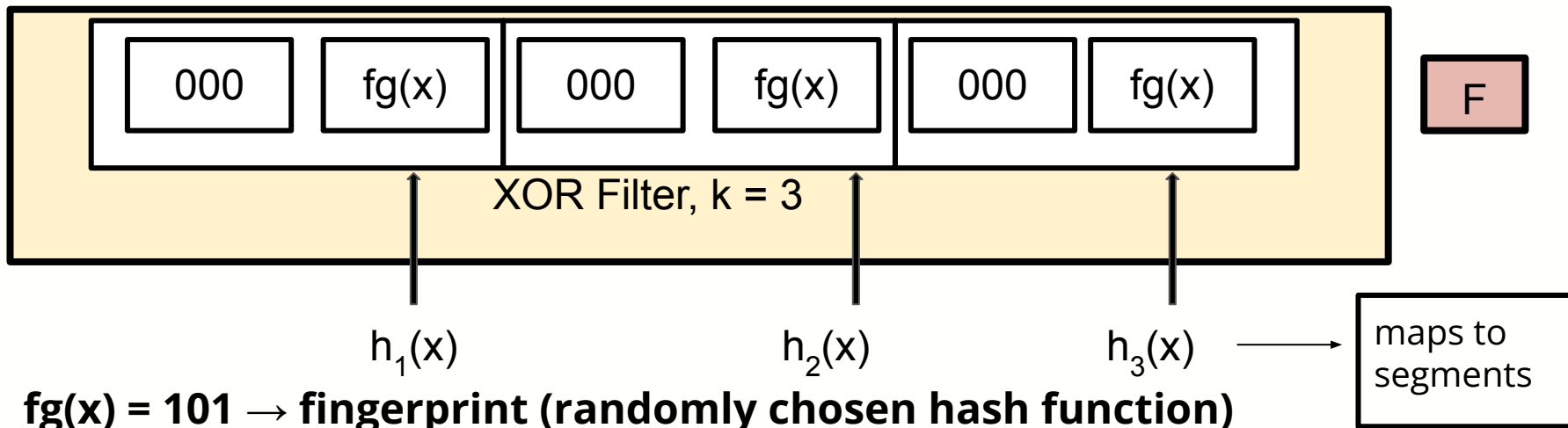
m = size of set
 k = # of hash funcs
 ϵ = false pos. rate
 ζ = blowup



$$F[h_1(x)] \text{ XOR } F[h_2(x)] \text{ XOR } F[h_3(x)] = \mathbf{fg(x)}$$

Binary-Fuse Filters (BFF) [ML22]

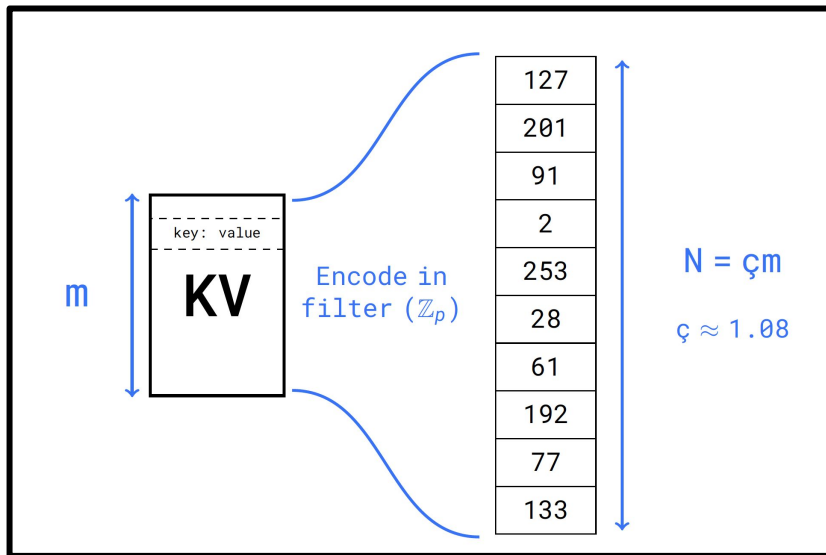
- The filter is an array with about $1.13 * |S|$ slots (for 3)
- Three hash functions h_1, h_2, h_3 map a key to three slots. A separate function $fp(x)$ gives the key's k -bit fingerprint



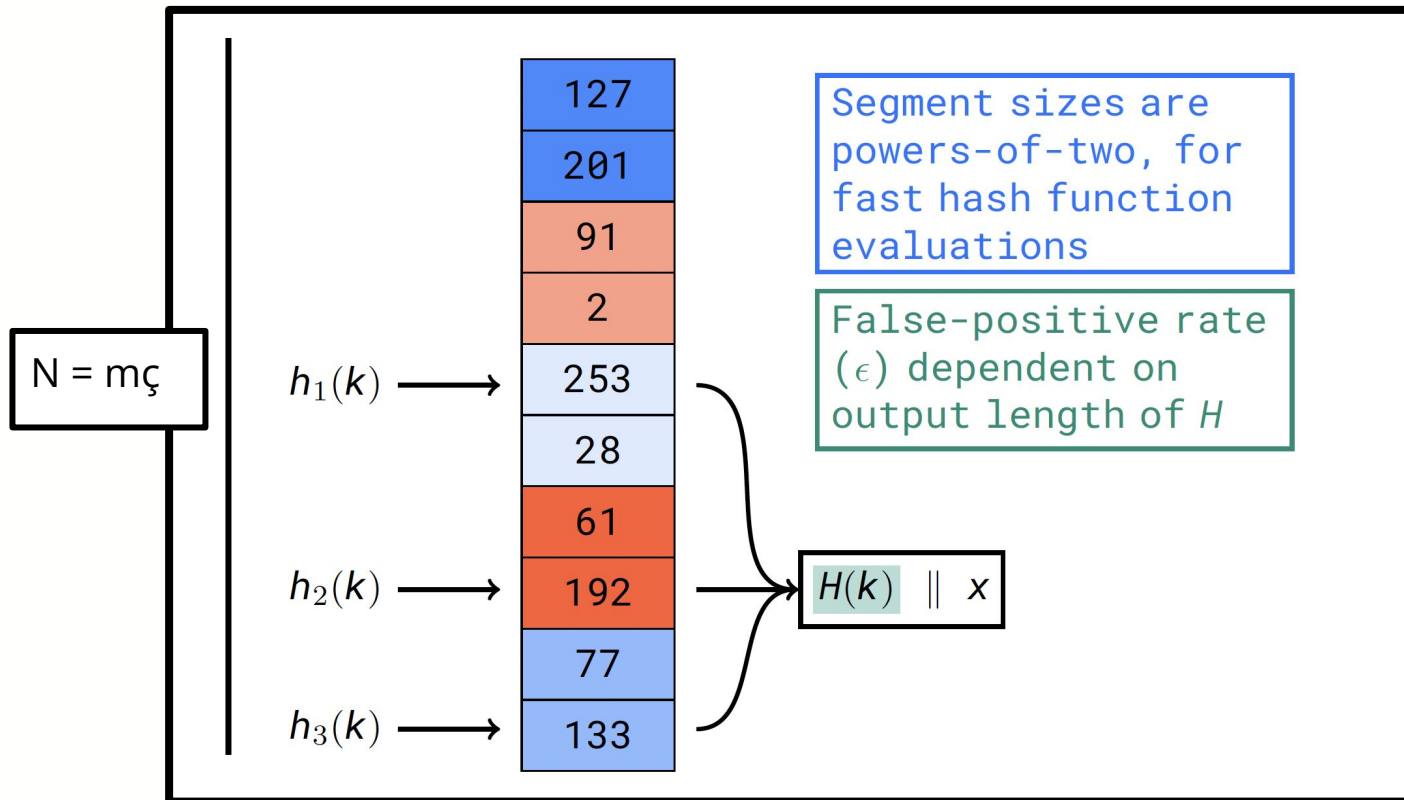
$$F[h_1(x)] \text{ XOR } F[h_2(x)] \text{ XOR } F[h_3(x)] = fg(x)$$

Binary-Fuse Filters (BFF)

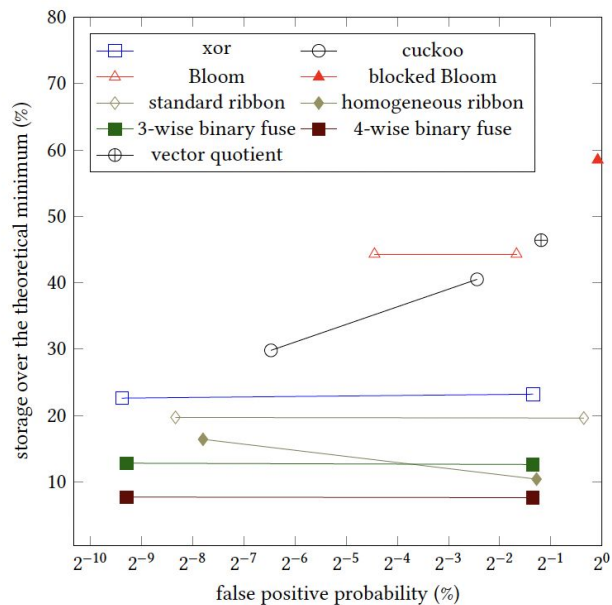
- $k \in \{3, 4\}$, constant space blowup reduced to $\zeta \in \{\approx 1.08, \approx 1.13\}$
- Break the filter into smaller consecutive segments:
 - $F = (F_1 \parallel \dots \parallel F_p)$, where P is chosen to be some value that ensures that each segment F_i contains 2^g entries, for some $g \in \mathbb{N}$



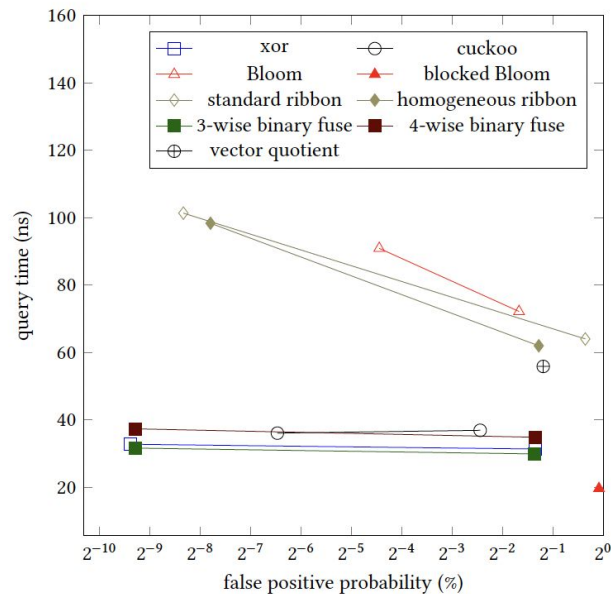
Binary-Fuse Filters (BFF)



Binary-Fuse Filters (BFF)



(a) Relative space usage

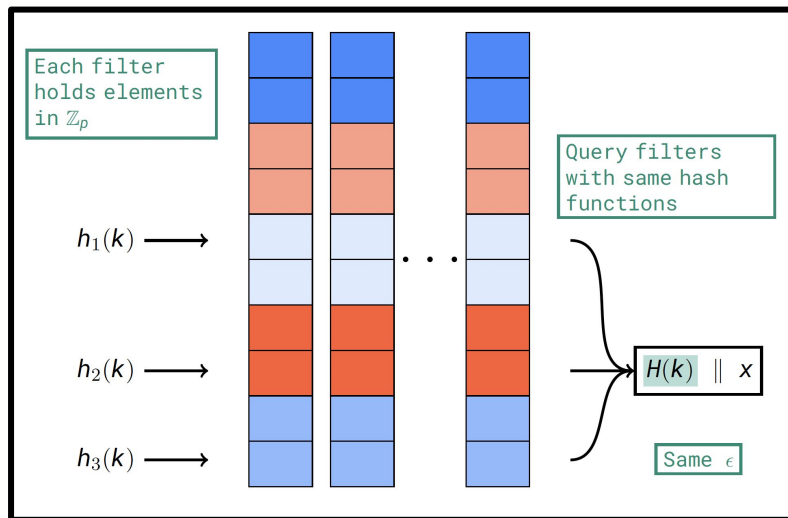


(b) Query time

Graphs taken from [ML22]

Binary-Fuse Filters (BFF)

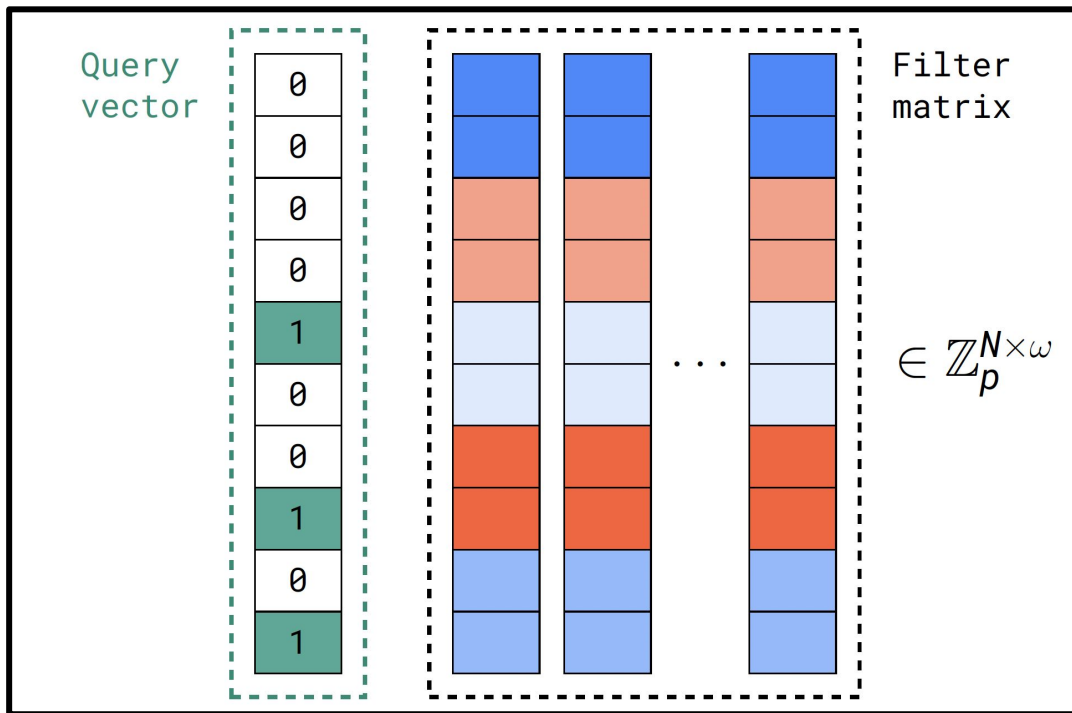
- We will store large values of the form: $key \parallel value$ (key-value maps)
 - Queries are performed over keys, responses return $key \parallel value$
- Filter holds elements in $\mathbb{Z}_p$: concatenated into a vector $F' = (F_1, \dots, F_d)$
 - Each F_j holds $\log(p)$ bits of data in every element for $j \in [d]$



$$fg(k, v) = \text{hash}(k) \parallel v$$

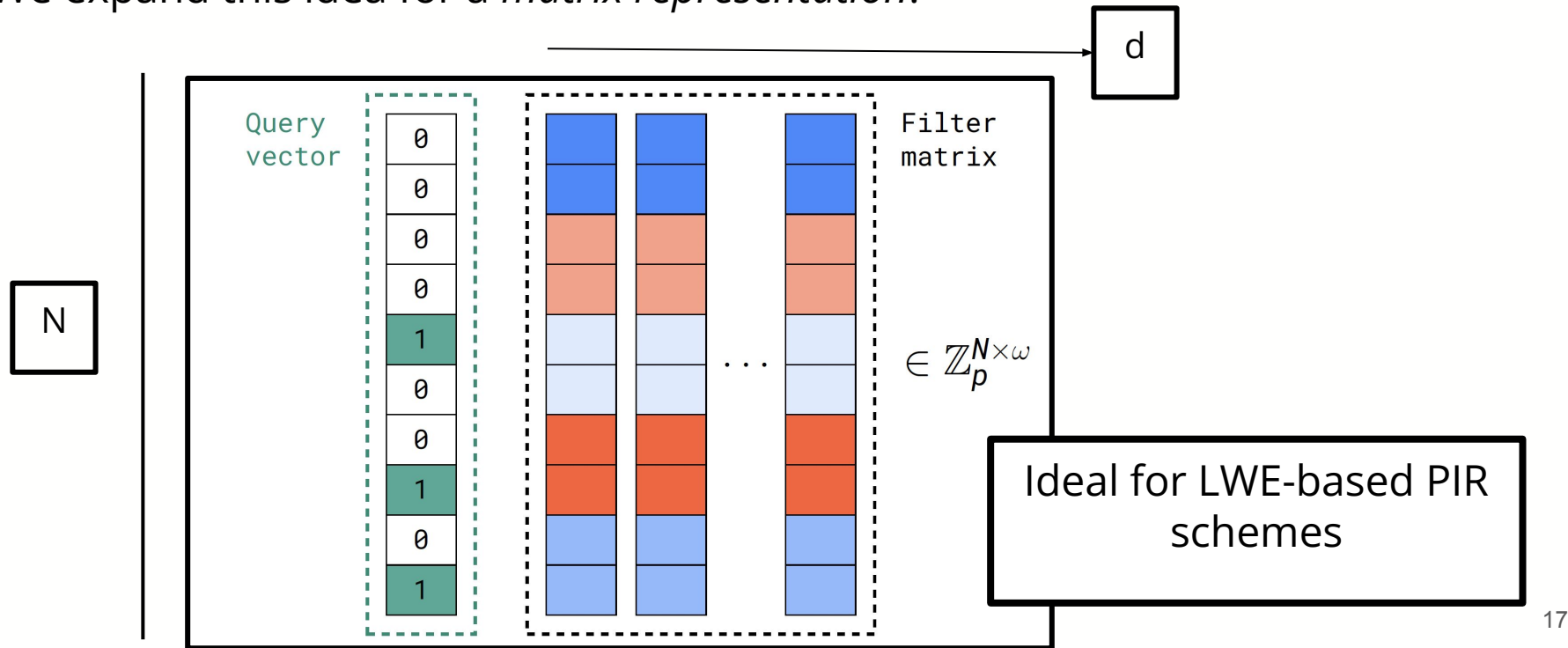
Binary-Fuse Filters (BFF)

We expand this idea for a *matrix representation*:



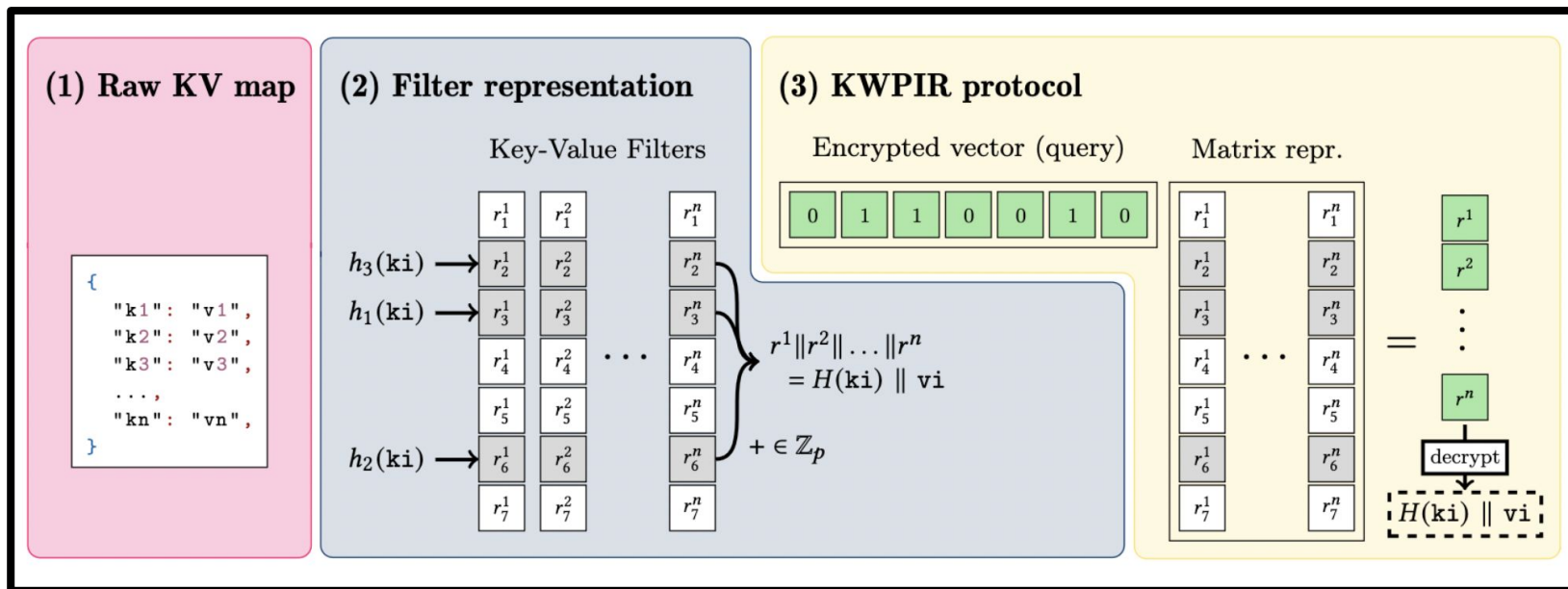
Binary-Fuse Filters (BFF)

We expand this idea for a *matrix representation*:



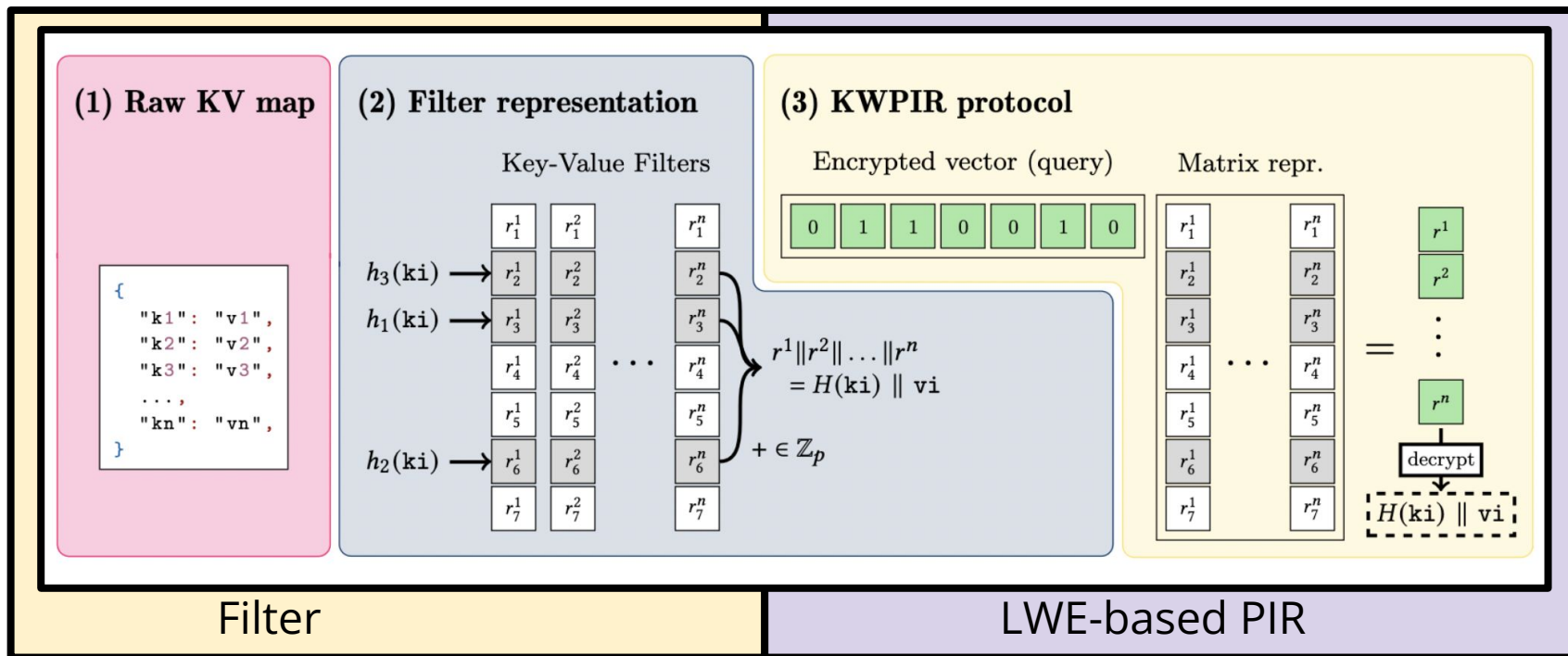
ChalametPIR [CD2024]

LWE-based single-server keyword-PIR scheme:

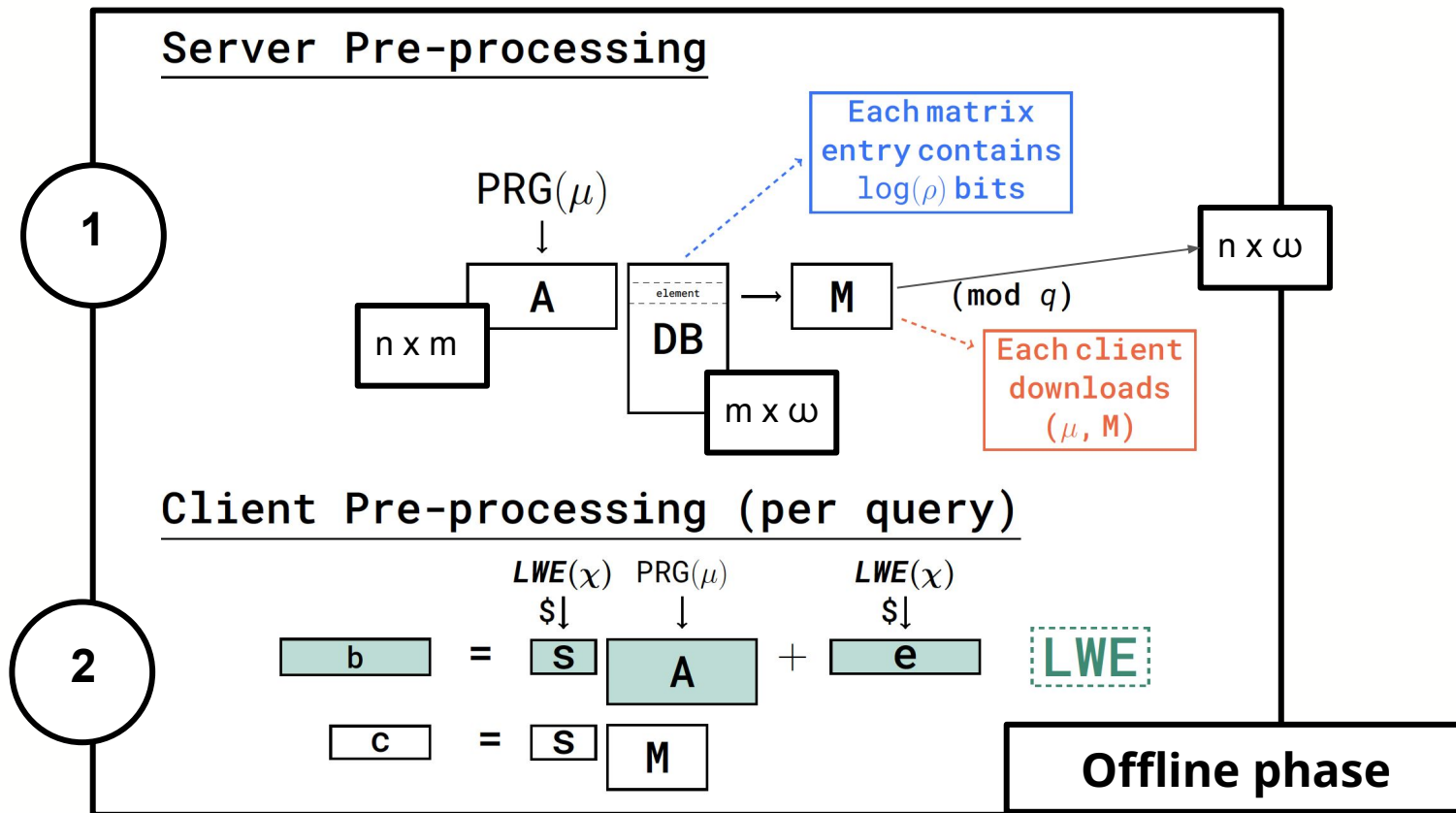


ChalametPIR [CD2024]

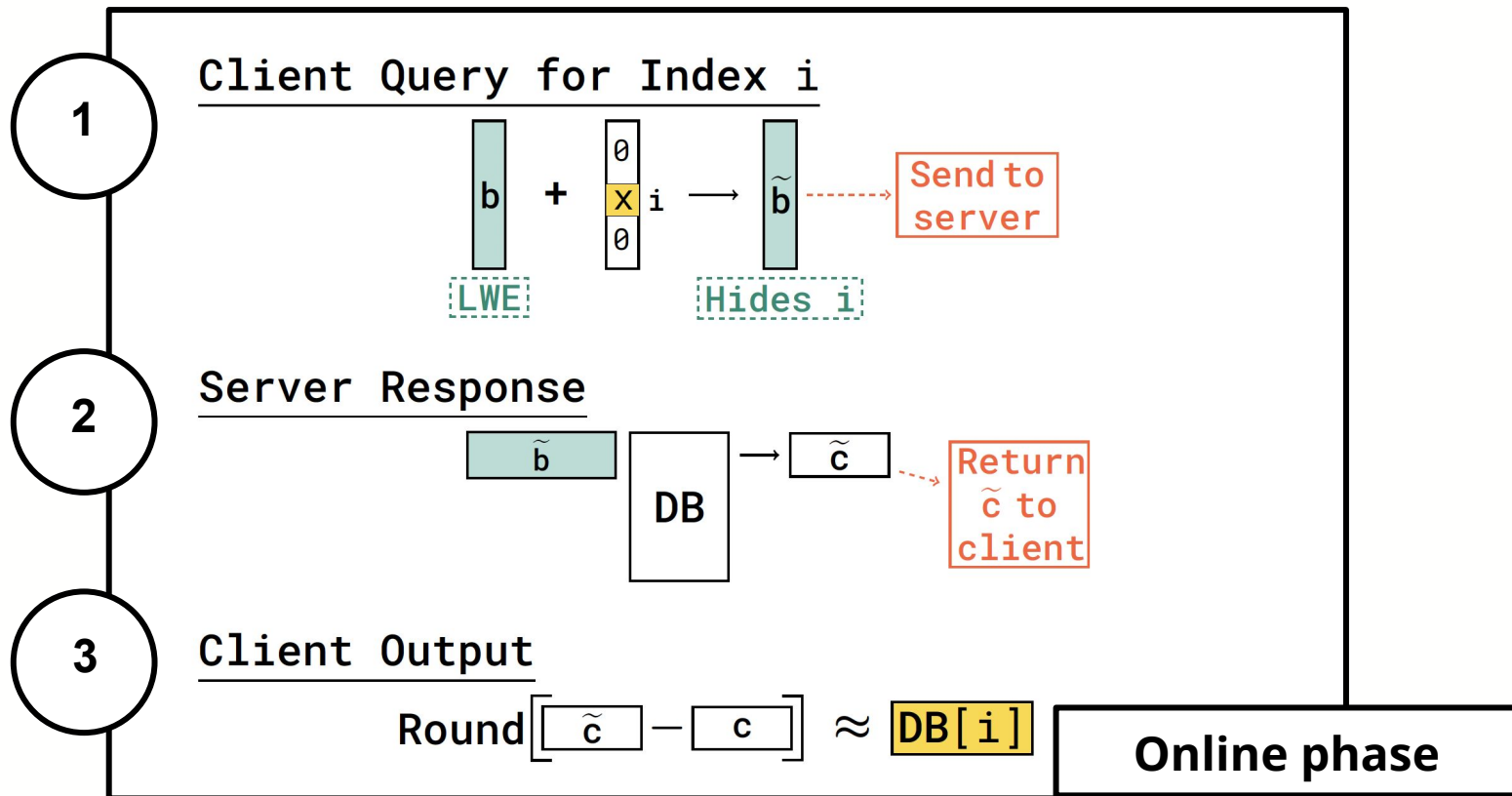
LWE-based single-server keyword-PIR scheme:



FrodoPIR (offline)

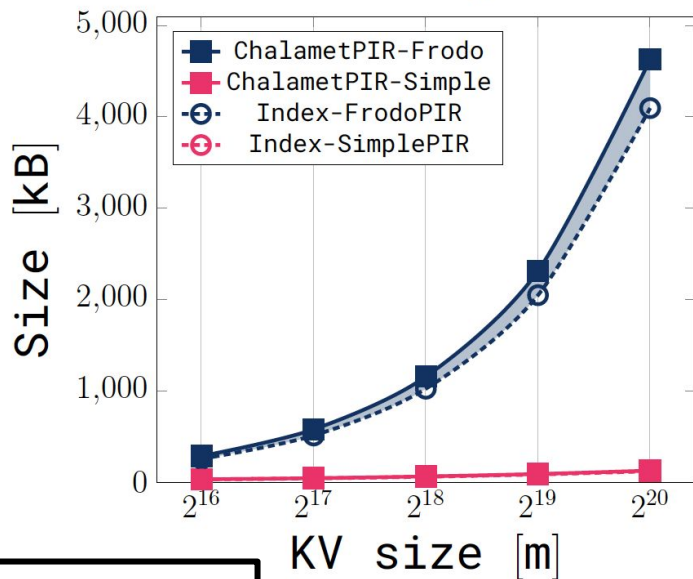


FrodoPIR (online)



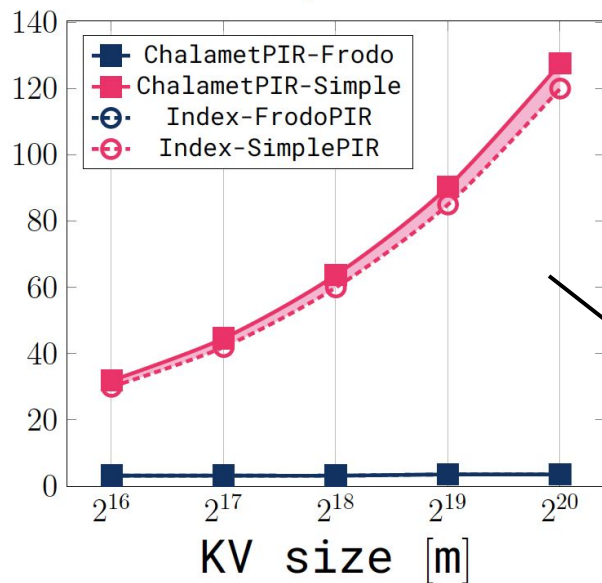
ChalametPIR [CD2024]

Query



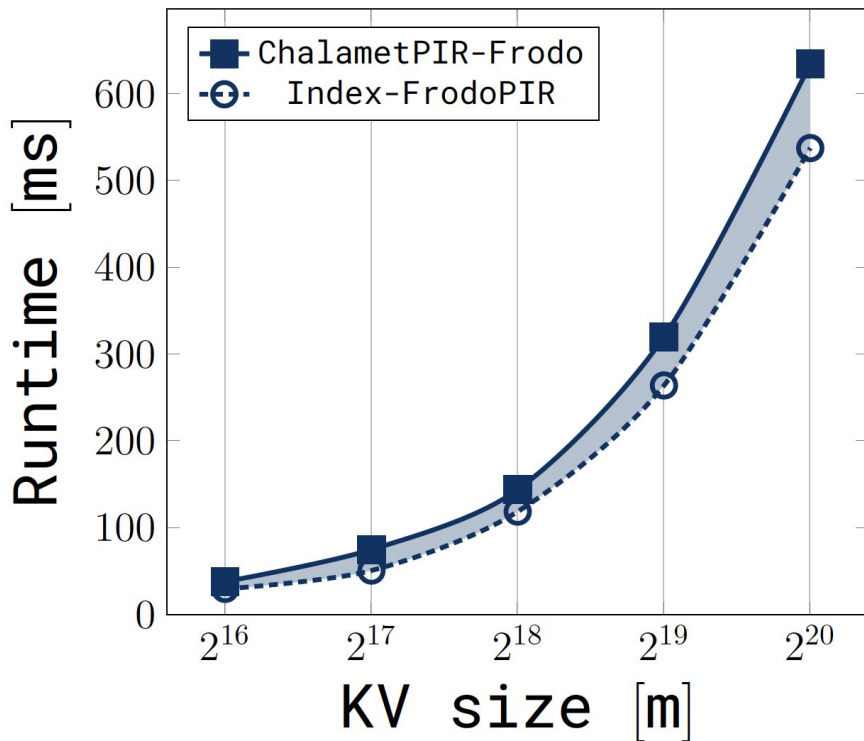
$$p = 2^{20} \quad q = 2^{32} \\ \lambda = 128$$

Response



Minimal overhead

ChalametPIR [CD2024]



$$p = 2^{20} \quad q = 2^{32} \\ \lambda = 128$$

Minimal
difference

But, it has downsides

- It makes updates even more difficult:
 - You have to update both the filter and the hint of the database
- How will one update?

In practice... not enough

- Databases are not structured in this simple way
 - **They are indexed by keywords** —————→
 - They are structured as JSON, Graphs, Excel spreadsheets
- The queries we are interested in are not simple:
 - Complex queries with AND/OR statements
 - Combination of database systems
 - Approximate nearest neighbor (ANN) elements
- Databases are constantly updated
- Is the security we assume enough?
 - What about malicious security?
 - What about private databases?

Ideas are coming!

**Not all systems are
created equally**

**Efficient:
index-based,
keyword-based,
public, static DB**

Recommendation systems

- The “most” used application of Information Retrieval systems:
 - Search engines
 - Recommendation systems: music, news, images ... everywhere
- Users expect accurate and almost near-instantaneous responses by *just* sending a short query
 - Engines store “snapshots”
 - constantly updated: rows change, rows are added/removed
 - for search engines: around 100–200 petabytes

- Fast rate of updates:
 - 2004 [NCO04]:
 - New pages are created at a rate of 8% per week, 320 million new pages added every week, roughly 3.8 terabytes in size
 - Only 20% of the pages available today will be still accessible after one year: high removal rate
 - After a year, about 50% of the content on the web is new → minor changes in place overall
 - It is a database that increases/decrease in size rapidly, but no significant changes “in place”:
 - If one page changed in the past, it will in the future → high predictability

- On queries [SJ04]:
 - Users query for one or two terms: in 2004, the mean was 2.4
 - Users launch:
 - Navigational queries, where accuracy highly matters
 - Informational queries, where reliability of source matters
 - Transactional queries
 - Google processes approximately 8.55 billion searches per day
- Allow for different distance: Euclidean, TF.IDF

Recommendation systems: search

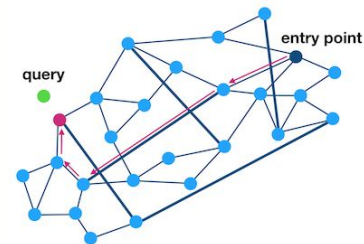
- What does it mean for PIR?
 - No research on PIR on this size of databases
 - Highly updatable in regards to increasing size, but not much on 'in-place'
 - Allow for schemes where adding/deleting rows is possible
 - Different user queries:
 - No need to batch for many queries
 - Accuracy is crucial for some queries
 - Cost remains to be analysed
- Search is changing due to web3: ML and follow-up queries

**We are at the start
of research on this**

Complex recommendation systems

2

- k-ANN
 - Real applications to recommendation systems
 - Graph-like structure that can be represented as a matrix
 - Multiple comparisons with neighbours to an entry point
- *Hierarchical navigable small world* (HNSW) [MY16] is used to improve semantic search performance:
 - Layered SW structure that increases accuracy



Recommendation systems

- What does it mean for PIR?
 - We can represent graph algorithms in the matrix structure used in PIR
 - But we need to obviously compare
 - But we need to obviously navigate the world (layers)
- Some systems can handle slow updates:
 - For some: the process of re-creating a database is slow still, so PIR preprocessing steps are not the burden
 - Ex: fine-tuned recommendation systems

**We are at the start
of research on this**

Pointers for research

- Expand the security model:
 - How does leakage impact it?
 - Is it attackable?
- Introduce 'updatable' techniques
- Look at other applications of DB: do we fulfil them?
- How do we deal with variable-length elements?
 - Is padding enough?
- Can we look at state-of-the-art data structures/graphs/matrix theory?

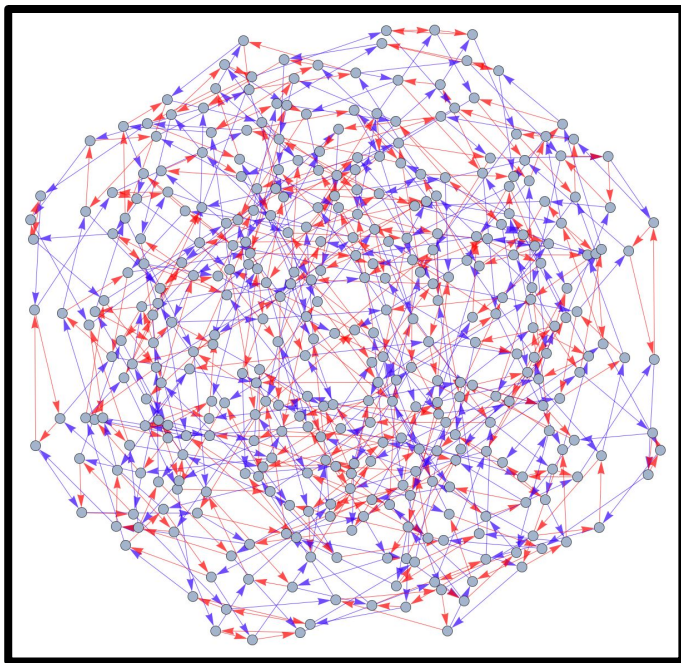
We are far from PIR for search engines, but we are starting!

Let's go back to the question

- Is this practical?
 - Kind of..
 - Works for some systems, but not for others
 - The communication is ok
 - The computation is ok
 - Financially, it might be valid
 - But it does not work for all applications out-of-the-box

Pointers for research

- Keyword-based PIR:
 - “Call Me By My Name: Simple, Practical Private Information Retrieval for Keyword Queries”: <https://eprint.iacr.org/2024/092>
 - “Binary Fuse Filters: Fast and Smaller Than Xor Filters”: <https://arxiv.org/abs/2201.01174>
- Security:
 - “Fully Malicious Authenticated PIR”: <https://eprint.iacr.org/2023/1804>
 - “VeriSimplePIR: Verifiability in SimplePIR at No Online Cost for Honest Servers”: <https://eprint.iacr.org/2024/341>
- Complex queries:
 - “Private Web Search with Tiptoe”: <https://eprint.iacr.org/2023/1438>
 - “Coeus: A System for Oblivious Document Ranking and Retrieval”: <https://eprint.iacr.org/2022/154>
- Updatability:
 - “Incremental Offline/Online PIR”
<https://www.cis.upenn.edu/~sga001/papers/incpir-sec22.pdf>



Kathryn Cramer, CC BY-SA 4.0
<<https://creativecommons.org/licenses/by-sa/4.0/>>, via Wikimedia Commons

THANK YOU!

@claucece

<https://sofiaceli.com/>

<https://eprint.iacr.org/2023/1063>

<https://github.com/brave-experiments/distefano>

References

- [KO97] Eyal Kushilevitz and Rafail Ostrovsky. *Replication is NOT Needed: SINGLE Database, Computationally-Private Information Retrieval*. In: 38th Annual Symposium on Foundations of Computer Science, FOCS '97, Miami Beach, Florida, USA, October 19-22, 1997. IEEE Computer Society, 1997, pp. 364–373. URL: <https://doi.org/10.1109/SFCS.1997.646125>
- [DPC23] Alex Davidson, Gonçalo Pestana, and Sofía Celi. *FrodoPIR: Simple, Scalable, Single-Server Private Information Retrieval*. In: Proc. Priv. Enhancing Technol. 2023.1 (2023), pp. 365–383. DOI: 10.56553/popets-2023-0022. URL: <https://doi.org/10.56553/popets-2023-0022>
- [Hen+23] Alexandra Henzinger et al. *One Server for the Price of Two: Simple and Fast Single-Server Private Information Retrieval*. In: 32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9-11, 2023. Ed. by Joseph A. Calandrino and Carmela Troncoso. USENIX Association, 2023. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/henzinger>

References

- [ML22] Thomas Mueller Graf and Daniel Lemire. 2022. *Binary Fuse Filters: Fast and Smaller Than Xor Filters*. ACM J. Exp. Algorithmics 27 (2022), 1.5:1–1.5:15. <https://doi.org/10.1145/3510449>
- [ML20] Thomas Mueller Graf and Daniel Lemire. 2020. *Xor Filters*. ACM J. Exp. Algorithmics 25 (2020), 1–16. <https://doi.org/10.1145/3376122>
- [PSY23] Sarvar Patel, Joon Young Seo, and Kevin Yeo. 2023. Don't be Dense: Efficient Keyword PIR for Sparse Databases. In 32nd USENIX Security Symposium (USENIX Security 23). USENIX Association, Anaheim, CA, 3853–3870. <https://www.usenix.org/conference/usenixsecurity23/presentation/patel>
- [CD24] Sofia Celi, Alex Davidson, Call Me By My Name: Simple, Practical Private Information Retrieval for Keyword Queries. In ACM CCS 2024. <https://eprint.iacr.org/2024/092>

References

- [NCO04] Ntoulas, A., Cho, J., and Olston, C. (2004). What's new on the Web?: The evolution of the web from a search engine perspective. In Proceedings of the 13th International World Wide Web Conference.
- [SJ04] Spink, A., and Jansen, B. J. (2004). A study of Web search trends. Webology.
- [MY16] Y. Malkov, D. Yashunin, Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs (2016), IEEE Transactions on Pattern Analysis and Machine Intelligence

THANK YOU!

@claucece

www.sofiaceli.com
